**Indian Institute of Technology Hyderabad**

Department of Artificial Intelligence

# Self-Healing Distributed Drone Swarm for Disaster Response

Coordination and task allocation
under drone and communication failures

Group Project Report

AI3403 Multi-Agent Systems

Course instructor: Dr. Venkata Renganathan

| | |
|---|---|
| Puli Dinesh (Team Lead) | AI23BTECH11019 |
| Sai Dhanush | AI23BTECH11021 |
| Ajay | AI23BTECH11011 |
| Nenavath Kalyan | CS23BTECH11039 |

November 2026

Public GitHub repository and experimental evidence

# Abstract

When a rescue drone fails, the remaining drones must find out what happened and resume its work using incomplete information. We built a two-dimensional simulator in which drones search a map, detect survivors, and exchange observations over a changing radio graph. They use local bids or a distributed optimizer to propose assignments. Majority leases control who can execute a task, and heartbeat messages help the swarm release unfinished work after a failure.

We evaluated 29 conditions using 20 seeds per condition, for 580 recorded episodes. With 20% scheduled drone failures, healing increased mission completion from 4 of 20 runs to 16 of 20. The strict recovery measure, which requires simultaneous reactivation of all the failed drone's unfinished tasks, was 96.25% without healing and 93.75% with healing. We investigated and retained this failed check. On a controlled graph, resilient consensus reduced estimation error under false reports. In the dynamic mission, distributed ADMM frequently fell back to the greedy auction. Severe radio degradation and the 100-drone runs exposed limits in finishing rescues despite high search coverage. These findings identify both where local coordination helped and where the design needs more work.

**Keywords** multi-agent systems, task allocation, ADMM, resilient consensus, self-healing, signal temporal logic.

# Contents

**Abstract** i

**1 Introduction** 1

1.1 Overview . . . 1
1.2 Problem and scope . . . 1
1.3 What we built . . . 1
1.4 Individual contributions . . . 1
1.5 Summary . . . 2

**2 Technical Background** 3

2.1 Overview . . . 3
2.2 Task allocation and distributed optimization . . . 3
2.3 Consensus with changing neighbors . . . 3
2.4 Safety and measurable requirements . . . 3
2.5 Summary . . . 3

**3 System Model** 4

3.1 Overview . . . 4
3.2 Environment and mission . . . 4
3.3 Agent state and communication . . . 5
3.4 Energy and faults . . . 5
3.5 Summary . . . 5

**4 Distributed Coordination** 6

4.1 Overview . . . 6
4.2 Search and shared estimates . . . 6
4.3 Assignment costs and constraints . . . 7
4.4 Distributed ADMM . . . 7
4.5 Ownership and recovery . . . 8
4.6 Safety and temporal monitoring . . . 8
4.7 Summary . . . 8

**5 Experimental Design** 9

5.1 Overview . . . 9

5.2 Scenario pairing and run provenance . . . 9
5.3 Measures and uncertainty . . . 10
5.4 Verification . . . 10
5.5 Summary . . . 10

**6 Results and Discussion** **11**
6.1 Overview . . . 11
6.2 Experiment 1. Allocation methods . . . 11
6.3 Experiment 2. Drone crashes . . . 12
6.4 Experiment 3. Radio degradation . . . 13
6.5 Experiment 4. Mission completion and strict recovery . . . 14
6.6 Experiment 5. Scaling . . . 15
6.7 Experiment 6. Resilient consensus . . . 16
6.8 Experiment 7. Ablations and sensitivity . . . 17
6.9 Verification and an observed handoff . . . 19
6.10 Summary . . . 19

**7 Conclusion and Next Steps** **20**
7.1 Overview . . . 20
7.2 What we learned . . . 20
7.3 Next steps . . . 20
7.4 Summary . . . 20

**A Reproduction and Supporting Figures** **21**
A.1 Overview . . . 21
A.2 Reproduction . . . 21
A.3 Controlled diagnostics . . . 21
A.4 Recovery and safety views . . . 23
A.5 Summary across conditions . . . 27
A.6 Summary . . . 27

**References** **28**

Chapter 1

# Introduction

## 1.1 Overview

A drone may detect a survivor and then lose its radio link or crash before it completes the response. If the other drones have different views of the situation, they may leave the task unattended or send several agents to do the same work. We ask whether a swarm can continue search and rescue after such failures using local information.

## 1.2 Problem and scope

Our setting is a simulated disaster area with obstacles, survivors and battery-powered drones. The swarm must cover the area, find survivors and assign a drone to each detected survivor. A communication link can disappear because an agent moves out of range or because the link fails. Messages may also be delayed or dropped. The distributed agents coordinate through current neighbors, and the simulator records the true state for evaluation.

We designed the project around three questions. First, how do local task bids and distributed optimization behave when the participants change? Second, can the swarm resume useful work after failures, and how does that affect mission completion? Third, how do radio degradation, unreliable estimates and larger populations limit performance? We use a centralized allocator as a comparison for the assignment problem. The physical model is planar and assumes ideal local proximity sensing, so the evaluation concerns the simulator.

## 1.3 What we built

The implementation includes a seeded environment, a range-based message bus, local map exchange, three allocation methods and a heartbeat recovery protocol. A local motion barrier enforces separation under the sensor and speed assumptions. A separate observer measures coverage, rescue completion, communication, energy and safety. Seven experiment suites test the design under matched scenarios. The repository includes the simulator, run configurations, CSVs, replay media and verification evidence [1].

## 1.4 Individual contributions

Table 1.1 follows our proposal's four-person allocation of responsibilities. Puli Dinesh served as team lead and also worked on simulation and evaluation.

**Table 1.1:** Individual responsibilities in the group project.

| Member and roll number | Primary area | Contribution |
|---|---|---|
| Puli Dinesh AI23BTECH11019 | Distributed optimization | ADMM formulation and task allocation, plus simulation integration and experimental evaluation. |
| Sai Dhanush AI23BTECH11021 | Consensus | Ordinary and resilient consensus, convergence analysis and comparison under faulty reports. |
| Ajay AI23BTECH11011 | Failure and self-healing | Heartbeat failure detection, redistribution of unfinished tasks and recovery measurement. |
| Nenavath Kalyan CS23BTECH11039 | Communication | Dynamic graph, link behavior, neighbor messaging and communication-based coordination. |

## 1.5 Summary

The project studies whether drones can coordinate search and rescue as agents and links fail. The next chapter introduces the ideas behind our choice of allocation, consensus and safety checks.

Chapter 2

# Technical Background

## 2.1 Overview

The swarm makes two kinds of decisions. It assigns work to individual drones and shares estimates of the mission state. These decisions have different goals, which helps explain why improving an estimate need not change which survivor is rescued.

## 2.2 Task allocation and distributed optimization

Multi-robot task allocation asks which robot should perform which job under costs and constraints [2]. Our costs reflect travel distance, battery, previous work and connectivity. A centralized linear program provides an assignment-cost reference. A local greedy auction proposes a feasible assignment quickly when enough capacity is available in its bid view. The alternating direction method of multipliers (ADMM) splits an optimization problem into local updates coupled by agreement constraints [3]. It gives us a way to compare fast bidding with iterative agreement on a fixed assignment problem.

## 2.3 Consensus with changing neighbors

In ordinary average consensus, agents repeatedly combine their current estimate with values received from neighbors. The spectrum of the update matrix governs the decay of disagreement on a fixed graph [4]. Metropolis weights can be computed from local degrees [5]. This is useful when the graph changes with motion. A malicious sender can still pull an ordinary average toward a false value. Weighted mean-subsequence-reduced (W-MSR) consensus discards up to a specified number of high and low reports relative to the receiver's value before averaging. Its resilience depends on connectivity and the assumed number of faulty neighbors [6].

## 2.4 Safety and measurable requirements

Signal temporal logic (STL) lets us express requirements over time and attach a signed robustness value [7]. A separation margin measures how much clearance remains above a threshold. A response margin measures whether an eligible drone gets close enough before a deadline. Monitoring a trace gives evidence about that trace. The motion controller enforces a local separation rule separately from the observer's measurement.

## 2.5 Summary

We use the assignment problem to compare allocation methods, consensus to share estimates and temporal requirements to evaluate behavior. The following chapter defines the environment, information and mission outcomes that these methods operate on.

Chapter 3

# System Model

## 3.1 Overview

The model captures travel, local sensing, battery use and changing communication with one-second steps. We specify the default scenario first and describe changes for individual experiments in Chapter 5.

## 3.2 Environment and mission

The world is $100 \times 100$ units on a grid of two-unit cells. Four random rectangular obstacles occupy roughly 8–12% of cells, and all free cells are reachable. The default instance has 25 search regions, nine stationary survivors and 20 point drones. A seeded hotspot gives the regions priorities from 1 to 3. A drone visits free cells and marks their centers as seen within six units. A survivor within that radius is detected with probability 0.9 per step. Figure 3.1 shows an example from the observer's viewpoint.

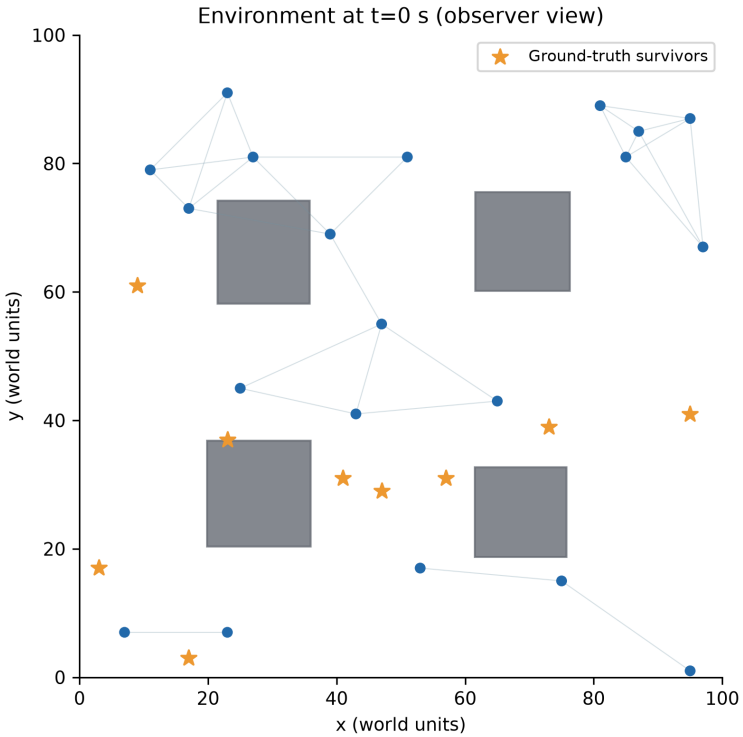



**Figure 3.1:** Recorded initial environment. The observer can see the survivor locations. Drones discover them through local sensing.

Search regions are complete at 95% local coverage. A response is complete when the assigned drone remains within 1.5 units of the survivor for three consecutive steps. The default mission ends after every survivor is rescued and at least 95% of the free-cell centers have been seen.

The horizon is 600 steps by default. Unfinished missions are recorded at their horizon with a timeout flag.

## 3.3 Agent state and communication

Each drone keeps its own position, velocity, remaining battery, local coverage map, detected survivors, selected task and recent neighbor information. A potential radio edge exists when two live drones are within the configured range. Links can fail at each step. The bus then applies packet loss and a delay of zero to two steps. Delivery requires an edge at delivery time as well as at transmission. Information may reach a distant drone through successive neighbor messages. The geometry is public and fixed. Survivor detections, bids and local maps spread through the bus.

**Table 3.1:** Default dynamics and communication settings.

| Parameter | Default |
|---|---|
| Step duration and speed cap | 1 s, 2 units/s |
| Sensing and radio ranges | 6 and 25 units |
| Link failure and packet drop probabilities | 0.02 and 0.05 |
| Initial battery and separation threshold | Uniform on $[60, 100]$ units, 2 units |
| Heartbeat timeout and allocation period | 5 and 20 steps |
| Response-arrival deadline | 100 steps after detection |

## 3.4 Energy and faults

The battery decreases by $0.025\|\boldsymbol{v}_i\| + 0.0001m_i + 0.012$ per step, where $m_i$ counts attempted directed edge transmissions. A drone stops sending and moving when it crashes. Injected crashes select a seeded subset of initial agents and schedule each crash uniformly in the first 60% of the horizon. A mission may end before a scheduled crash, so we report observed and scheduled counts separately. A different fault model in Experiment 6 changes the scalar estimate transmitted by one agent while that agent continues flying.

## 3.5 Summary

The world model defines observable search, response and failure events. Local mission data are exchanged over radio, while the observer uses the complete simulated state to measure performance.

Chapter 4

# Distributed Coordination

## 4.1 Overview

Each step connects sensing, communication and motion. Drones search for unseen cells, share discoveries, update their estimates and decide which tasks they can execute. A failure feeds back into the same allocation process. Figure 4.1 gives the main stages.

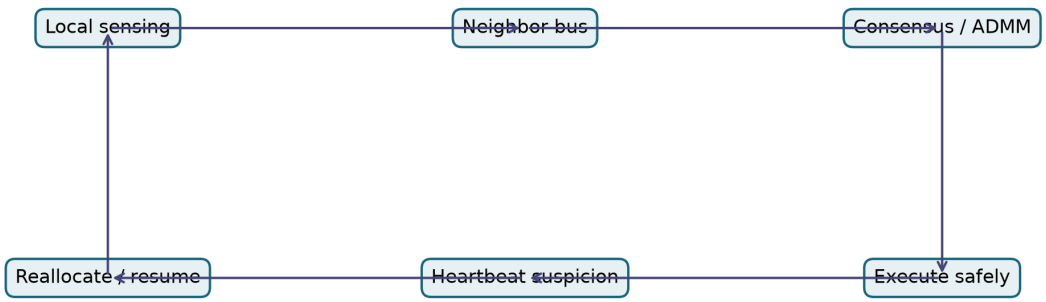



**Figure 4.1:** Coordination pipeline. Each controller uses local observations and neighbor messages. The recovery path returns unfinished work to allocation.

## 4.2 Search and shared estimates

Drones choose unseen frontier cells in their assigned region when possible. Unassigned drones can explore without claiming a region. When direct motion is blocked, A* on the known obstacle map provides a detour. Received maps are combined with the local seen mask. This preserves discoveries as information propagates through the graph.

For a fixed connected graph, the ordinary consensus test uses the iteration

$$z_i^{k+1} = z_i^k + \alpha \sum_{j \in \mathcal{N}_i} a_{ij}(z_j^k - z_i^k), \qquad a_{ij} = \frac{1}{1 + \max\{k_i, k_j\}}. \tag{4.1}$$

Here $\mathcal{N}_i$ is the neighbor set and $k_i$ is its size. The spectral check uses $\alpha = 1$ and compares the disagreement decay with the largest non-consensus eigenvalue magnitude of the Metropolis matrix. The mission uses relaxed weights and local innovations because each drone's observations change over time. W-MSR trims up to $F$ values above and up to $F$ values below the drone's own value before averaging. The controlled fault experiment sets $F = 1$ on a seven-node complete graph checked for $(2, 2)$-robustness. The estimates describe shared mission state. The allocation policy uses communicated maps and bids.

## 4.3 Assignment costs and constraints

At an allocation round, let $i$ index candidate drones and $j$ index available search or response tasks. The cost matrix uses the proposal's five factors:

$$c_{ij} = d_{ij} + \frac{20 d_{ij}}{\max(b_i, 1)} + 2L_i + \frac{3}{k_i + 1} - 5p_j. \tag{4.2}$$

The straight-line distance $d_{ij}$ estimates travel effort. The second term makes long trips more expensive for drones with low battery $b_i$. Workload $L_i$ counts completed responses, so one past response adds two cost units. The connectivity penalty is three for a drone with no neighbors and falls with degree $k_i$. The priority $p_j$ belongs to the task. For any given task, it shifts every candidate's cost equally. It therefore changes neither the auction winner nor the gap between its two lowest bids. It also adds a constant to the objective when all task columns are assigned. The fixed weights $(1, 20, 2, 3, 5)$ are heuristic design choices shared by the allocation methods. The experiments do not identify optimal weights.

The fractional problem uses an assignment $X = [x_{ij}]$, a cost matrix $C = [c_{ij}]$ and capacities $q_i$:

$$\begin{aligned} \underset{X}{\text{minimize}} \quad & \sum_i \sum_j c_{ij} x_{ij} \\ \text{subject to} \quad & \sum_i x_{ij} = 1 \qquad \text{for each task } j, \\ & \sum_j x_{ij} \le q_i \qquad \text{for each drone } i, \\ & 0 \le x_{ij} \le 1 \qquad \text{for all } i, j. \end{aligned} \tag{4.3}$$

The centralized baseline solves this problem with ideal global information. The greedy auction sorts tasks by the gap between the best and second-best bids, largest gap first. It assigns each task to the cheapest candidate with remaining capacity. Task and drone order break ties. A large gap signals that using the second choice is comparatively costly. This greedy ordering does not certify a global optimum.

## 4.4 Distributed ADMM

Each participant keeps a local copy $X^{(i)}$ of the assignment. Only row $i$ contributes to its local objective. Copies agree over graph edges through edge variables $Z_{ij}$ and scaled duals $U_{ij}^{(i)}$. Let $\mathcal{P}$ be the feasible set in Equation 4.3, and let $C_i$ contain row $i$ of $C$ and zeros elsewhere. The reference update at a node with neighbors is

$$\begin{aligned} X^{(i),+} &= \Pi_{\mathcal{P}} \left( \frac{1}{|\mathcal{N}_i|} \sum_{j \in \mathcal{N}_i} (Z_{ij} - U_{ij}^{(i)}) - \frac{C_i}{\rho |\mathcal{N}_i|} \right), \\ Z_{ij}^{+} &= \tfrac{1}{2} \left( X^{(i),+} + X^{(j),+} \right), \\ U_{ij}^{(i),+} &= U_{ij}^{(i)} + X^{(i),+} - Z_{ij}^{+}. \end{aligned} \tag{4.4}$$

Projection alternates between column simplices and nonnegative row capacity sets with Dykstra corrections. The primal residual measures disagreement with $Z$, and the dual residual measures changes in $Z$ scaled by $\rho$. In a mission round, nodes exchange copies when their epoch, task list and candidate identities match. The default penalty is $\rho = 4$, residual tolerance $10^{-3}$ and local budget 12 updates. The auction supplies proposals while ADMM runs. Accepted ADMM rounds produce rounded proposals, and the run records explicit fallback when a round fails or is replaced before convergence.

## 4.5 Ownership and recovery

Agents periodically advertise heartbeats. A receiver suspects a neighbor after five missed steps in the default setting. Sequence-numbered updates spread the suspicion and the release of unfinished work. A later message can clear a suspicion. Drones use their own recent neighbor history for relay moves, so a spare drone may move toward a last known position without knowing the whole partition.

A vote in epoch $k$ can authorize execution in epoch $k + 1$. Each initial identity can vote for one candidate per task per epoch. A certificate needs more than half of the initial population. Two majorities must intersect, so an honest voter cannot help certify conflicting owners for the same task and epoch. Leases expire at the shared boundary. The design assumes honest lease voting and a common clock. A minority partition can continue exploring but may have to wait before claiming a task.

## 4.6 Safety and temporal monitoring

Each drone has an ideal onboard proximity observation within 6.05 units at default settings. This radius exceeds $d_{\min} + 2v_{\max}$, so an unseen drone cannot close the separation gap in one step. For a visible peer, the controller constrains motion using a supporting halfspace outside that peer's next-step reachable ball. Infeasible or blocked movement results in a hold. The observation supplies nearby positions independently of radio messages.

The observer evaluates two discrete-time STL requirements. For separation it records the minimum over sampled steps and live pairs:

$$r_{\text{sep}} = \min_{t} \min_{i<j:\, i,j \text{ live}} \big(\|\boldsymbol{p}_i(t) - \boldsymbol{p}_j(t)\| - d_{\min}\big). \tag{4.5}$$

For response it checks whether an eligible assigned drone reaches 1.5 units within 100 steps of detection. The window margin is the maximum of $1.5 - d$ over eligible arrivals. An unassigned window uses a categorical penalty. Pending windows remain separate from deadline violations. Arrival and the three-step rescue dwell are measured separately.

## 4.7 Summary

Local sensing and message exchange generate task proposals. Majority leases resolve conflicting execution decisions, and the motion barrier constrains each move. The evaluation separates the behavior of these mechanisms from their effect on mission success.

Chapter 5

# Experimental Design

## 5.1 Overview

We varied one part of the system at a time where the experiment supports that comparison. Every reported condition has 20 recorded seeds, and we keep failed runs in the analysis.

## 5.2 Scenario pairing and run provenance

The evaluation uses seeds 3001–3020 across 29 conditions, yielding 580 episodes. Compared methods share the seeded world, survivors, initial positions, batteries and intervention schedule where applicable. Environment, failures, sensing, links, bus loss and faulty reports use separate random streams. Once trajectories differ, their local observations and links can differ as part of the outcome. Calibration used seeds 41–43. Some evaluation-series scenarios appeared in earlier smoke checks, so this is not a fully blind holdout. The objective weights were retained throughout. Final evaluation outputs were rerun after correcting the same-step recovery boundary and per-survivor dwell reset.

**Table 5.1:** Seven evaluation suites. Each condition has 20 seeds.

| Suite | Question | Conditions | Runs |
|---|---|---|---|
| 1 | Allocation | Centralized LP, auction and ADMM | 60 |
| 2 | Drone failure | 0, 10, 20 and 30% scheduled crashes | 80 |
| 3 | Communication | Nominal, degraded and severe radio settings | 60 |
| 4 | Self-healing | On versus off at 20% scheduled crashes | 40 |
| 5 | Scaling | 10, 20, 50 and 100 drones on the same map size | 80 |
| 6 | Faulty reports | Ordinary versus W-MSR under bias or random reports | 80 |
| 7 | Ablation and sensitivity | Full system, three component removals and five parameter variants | 180 |

The crash and healing comparisons use a 300-second horizon. The 10-drone scaling case uses 900 seconds, with 600 seconds for the other populations. The radio comparison varies range, link failures and drops together. From nominal to severe, $(\text{range}, p_{\text{link}}, p_{\text{drop}})$ changes from $(25, .02, .05)$ to $(20, .15, .15)$ and $(15, .30, .30)$. Experiment 6 isolates the scalar misinformation on a complete seven-drone graph with no delay or drop. Experiment 7 reports component removals and the recorded sensitivity configurations. Some parameter variants also differ from the full ADMM/W-MSR configuration, so comparisons with the full system are descriptive.

## 5.3 Measures and uncertainty

We report mission success as the fraction finishing by the horizon. Mean recorded time includes the horizon for timeouts. It is a capped summary, with different caps stated explicitly. Coverage is the fraction of free cells seen and detection and rescue are fractions of survivors. Energy is initial minus remaining battery summed over agents. Communication counts directed transmission attempts, including dropped packets. It does not measure bytes. Workload balance uses Jain's index on distance traveled, $J = (\sum_i w_i)^2/(N \sum_i w_i^2)$, over the initial population.

Recovery is measured from a crash to the first step when every unfinished task held at failure is simultaneously selected by a live owner. An empty task set qualifies at the crash step. An unrecovered event keeps its censoring time and an explicit flag. Consensus error uses the initial honest mean in controlled trials and true observer coverage for mission coverage estimates. We also record separation margin, response deadline violations and duplicate execution.

For each condition, we report sample means, standard deviations and approximate two-sided 95% Student $t$ intervals over seeds. Intervals for fractions can extend outside $[0, 1]$, and a constant sample yields a zero-width interval. The required Welch tests compare the allocation groups descriptively. They ignore the scenario pairing and multiple comparisons, and are not a survival analysis. No claim depends on treating overlapping conditions across suites as independent replications.

## 5.4 Verification

Tests cover the assignment-cost reference, theoretical consensus rate, graph robustness, local redistribution, separation, STL signs, object isolation and deterministic replay. Debug mode checks coverage and battery monotonicity, obstacle exclusion, dead-agent behavior and bus counts. The verification record reports 31 passing tests, all 580 expected evaluation rows, and matched scenario groups. It separately retains the strict recovery scientific failure.

## 5.5 Summary

Pairing scenarios makes method comparisons interpretable. The capped time and success fraction answer different questions, and strict recovery measures task reactivation rather than overall completion. These distinctions are used throughout the results chapter.

Chapter 6

# Results and Discussion

## 6.1 Overview

Healing increased the fraction of missions that finished after crashes. The controlled ADMM solver matched its reference, but mission rounds frequently fell back to auction. Robust consensus reduced estimate error, while mission outcomes stayed the same in the fault comparison. This chapter reports what each comparison supports.

## 6.2 Experiment 1. Allocation methods

**Table 6.1:** Experiment 1 outcomes from 20 runs per method. Time includes the timeout horizon.

| Method | Finished | Time (s) | Energy | Attempts | Jain index |
|---|---|---|---|---|---|
| Centralized | 19/20 | 89.00 | 77.03 | 8,598.2 | 0.976 |
| Auction | 20/20 | 113.95 | 126.91 | 9,430.0 | 0.995 |
| ADMM | 20/20 | 113.95 | 126.91 | 9,430.0 | 0.995 |

Table 6.1 reports energy in battery units and communication as the mean total transmission attempts per episode. The centralized baseline finished 19 of 20 runs and both distributed methods finished 20 of 20. Mean recorded times were 89.00 seconds for centralized allocation and 113.95 seconds for auction and ADMM (Figure 6.1). The Welch comparison for capped time gives $p = 0.375$ for centralized versus each distributed method. It does not establish a difference in mean capped time. Centralized allocation used less total battery in these runs ($p \approx 3.24 \times 10^{-5}$ in the recorded Welch comparison). Mission length and flight decisions also influence energy use.

Auction and ADMM produced identical completion times in all 20 matched cases. In the ADMM condition, 26,867 of 27,267 started local rounds explicitly fell back to auction, about 98.53%. Some rounds can remain in progress when a mission ends. We did not measure a mission-time benefit from adding ADMM. The fixed connected test in Figure A.1 ends with an objective gap of about $2.35 \times 10^{-7}$ relative to the LP. This tests convergence on a fixed instance.

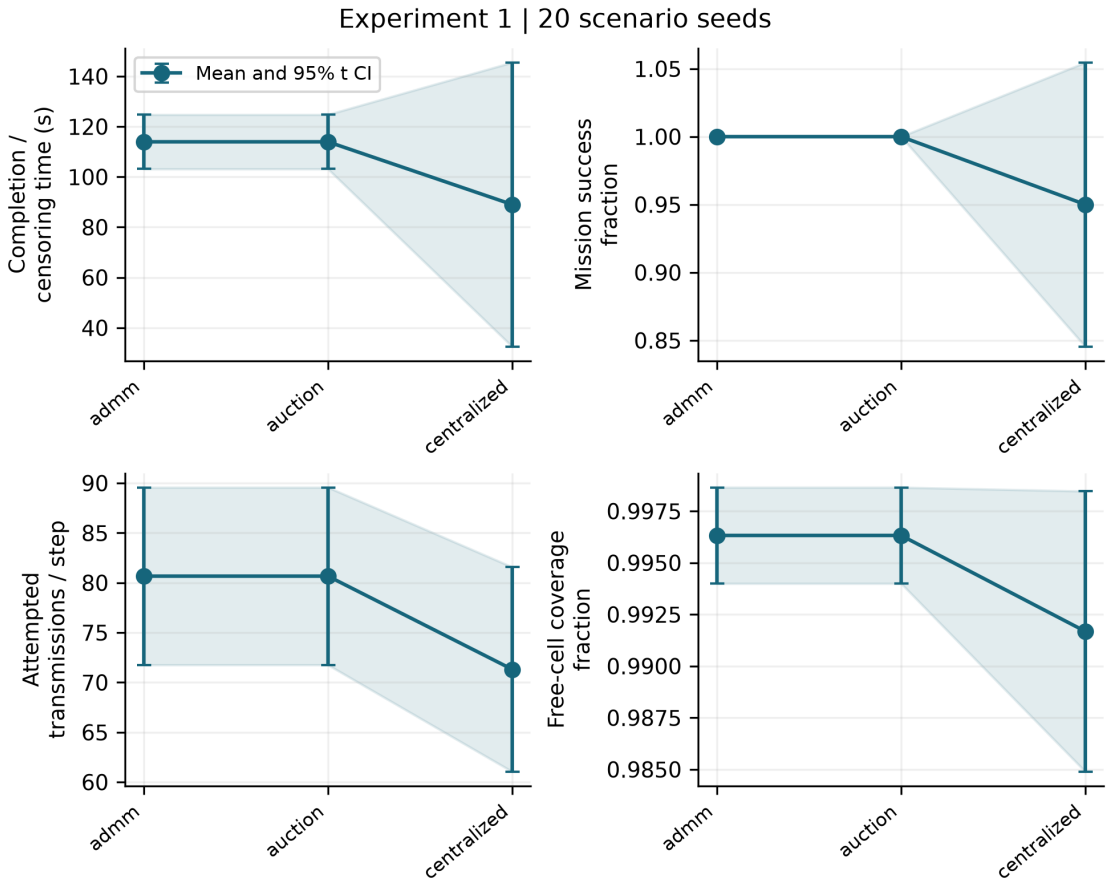



**Figure 6.1:** Allocation comparison over 20 seeds per condition. Time includes timeout horizons. Source: Experiment 1 CSV.

## 6.3 Experiment 2. Drone crashes

At 0, 10, 20 and 30% scheduled failures, success was 100, 100, 80 and 80%, respectively. Mean recorded time increased from 113.95 seconds to 199.90 seconds (Figure 6.2). The 10, 20 and 30% groups observed 33 of 40, 66 of 80 and 105 of 120 scheduled crashes. These counts account for missions ending before later crashes. At 20 and 30%, the sample success fractions were equal and time varied considerably across scenarios.

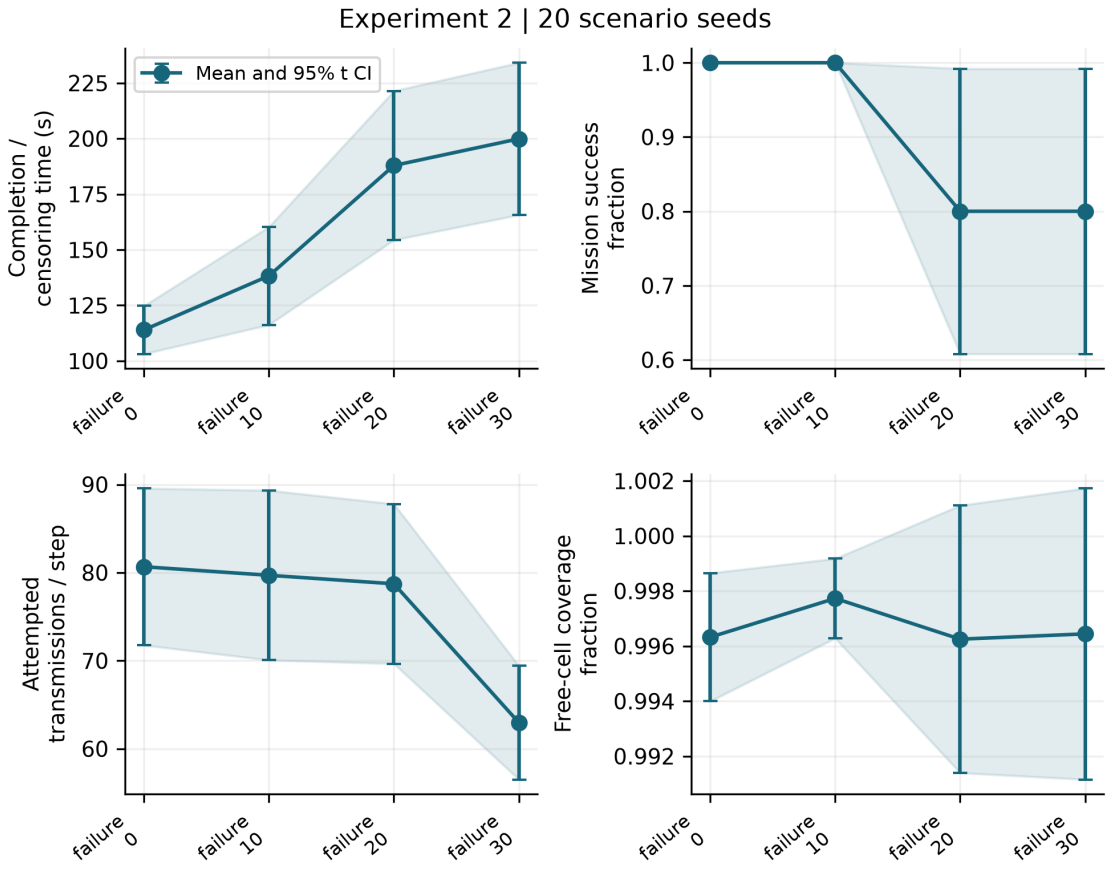



**Figure 6.2:** Crash sweep. The success fraction drops at 20% scheduled failures. Source: Experiment 2 CSV.

## 6.4 Experiment 3. Radio degradation

Under nominal, degraded and severe radio settings, 20, 13 and one of 20 missions finished, respectively. The severe case still had near-complete mean coverage at the horizon (Figure 6.3). Drones can keep exploring when they struggle to exchange assignments. The combined sweep cannot isolate the separate effects of range, link failures and packet drops.

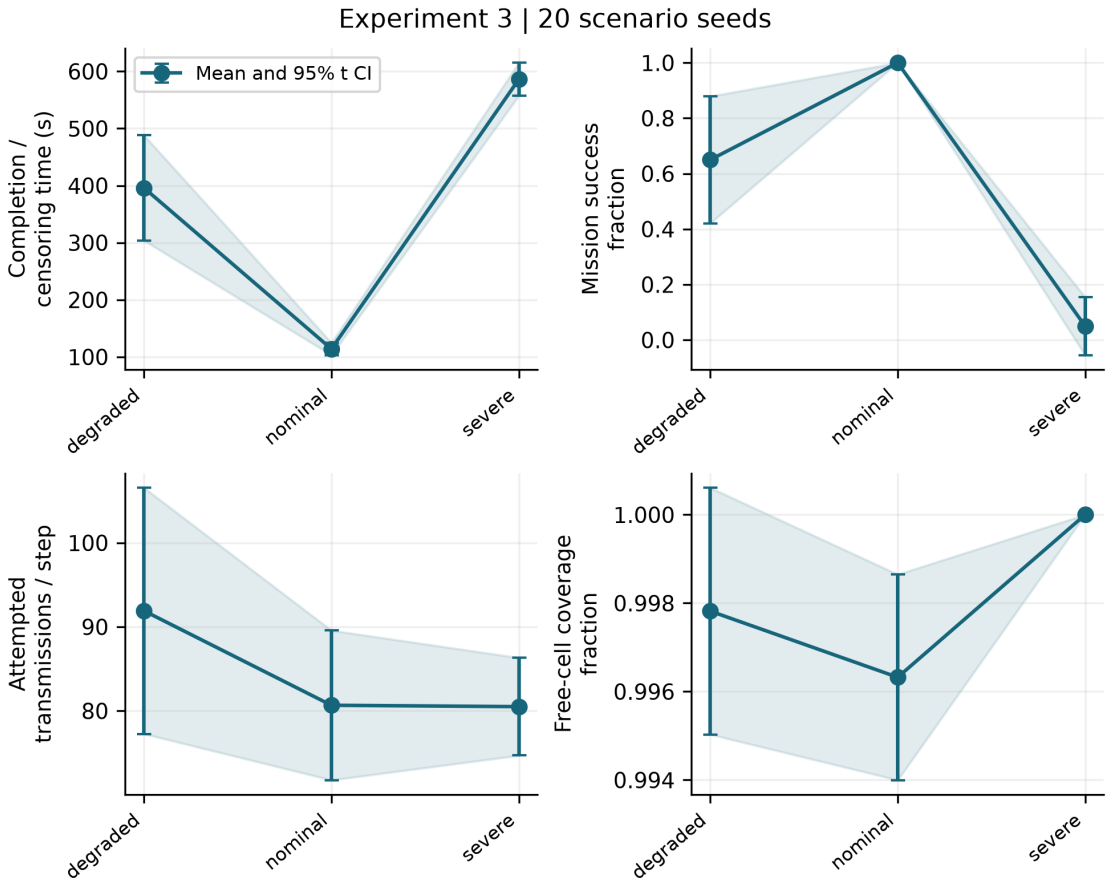



**Figure 6.3:** Radio conditions change mission success despite high observed coverage. Source: Experiment 3 CSV.

## 6.5 Experiment 4. Mission completion and strict recovery

At 20% scheduled failures, healing increased mission success from 4 of 20 to 16 of 20. Mean recorded time decreased from 259.85 to 187.90 seconds (Figure 6.4). The strict per-episode recovery score moved in the other direction, from 96.25% without healing to 93.75% with healing. The paired difference was −2.50 percentage points with an approximate 95% interval of $[-7.73, 2.73]$ percentage points. The expected improvement in strict recovery did not pass the sanity check.

**Table 6.2:** Observed failures by work held at the failure step in Experiment 4.

| Condition | Crashes | Empty sets | Nonempty sets | Nonempty recovered |
|---|---|---|---|---|
| Healing off | 73 | 68 | 5 | 2 |
| Healing on | 66 | 60 | 6 | 2 |

Most failures had no unfinished work and qualified for immediate recovery. We audited seed 3012 in the healing-on group. Drone 2 crashed at step 31 while holding region R16, which had 80.56% coverage at that point. Other flights took R16 above 95% at step 53. No live drone selected R16 as its current task after the crash, so that event stayed unrecovered by the strict rule even though the mission finished at step 212. This explains how task reactivation and mission success can diverge in a specific trace. The event counts are too small to establish a general improvement in nonempty handoffs.

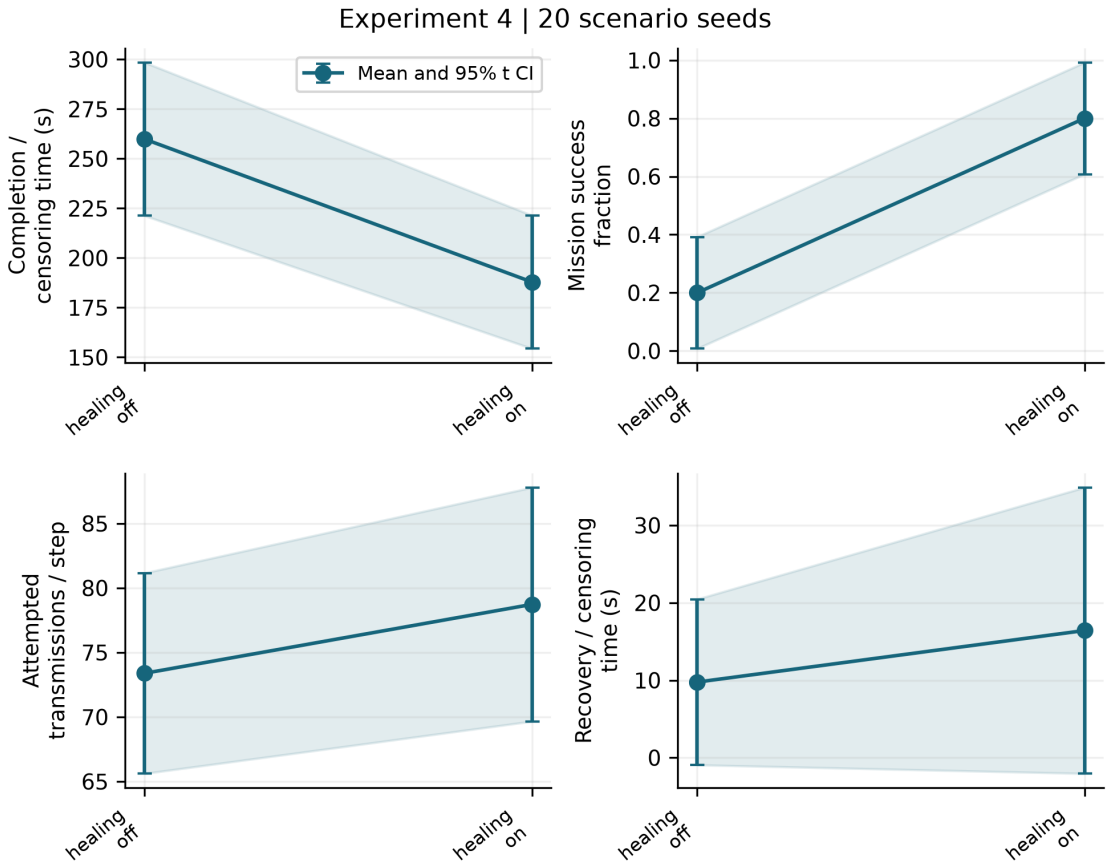



**Figure 6.4:** Healing comparison. The bottom-right panel is mean recorded recovery time. The text gives strict recovery rates. Source: Experiment 4 CSV.

## 6.6 Experiment 5. Scaling

For 10, 20, 50 and 100 drones, mission success was 95, 100, 85 and 45%, respectively. Figure 6.5 uses the source figure's alphabetical condition order. Mean transmission attempts per step increased from 22.72 at 10 drones to 1,782.32 at 100. All 11 timed-out 100-drone episodes reached the coverage target and detected every survivor, but left rescues unfinished. We have not isolated whether assignment availability, local movement constraints or another factor dominated these stalls. The different 10-drone horizon also matters when comparing raw times.

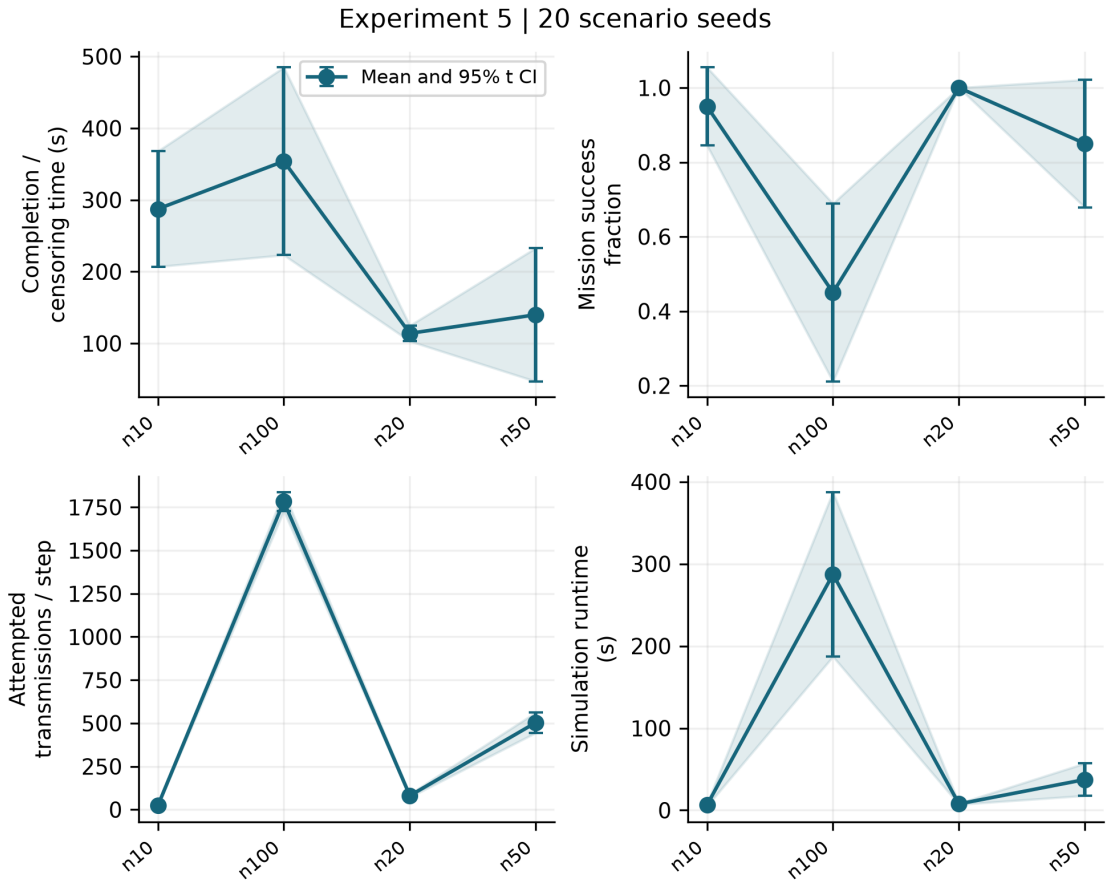



**Figure 6.5:** Scaling on a fixed map. The 10-drone condition has a longer horizon. Source: Experiment 5 CSV.

## 6.7 Experiment 6. Resilient consensus

Under bias, controlled mean honest-reference error dropped from 9.49732 with ordinary consensus to 0.04352 with W-MSR. Under random reports it dropped from 3.27952 to 0.01606. The respective mission coverage-estimate errors were 0.96900 and 0.32464 for ordinary consensus and about 0.00947 in both W-MSR conditions. The controlled curves appear in Figure A.4. All four mission conditions finished 19 of 20 runs with the same mean recorded time of 169.90 seconds. Allocation uses local maps and bids, which explains why a better scalar estimate did not change assignment decisions in this test.

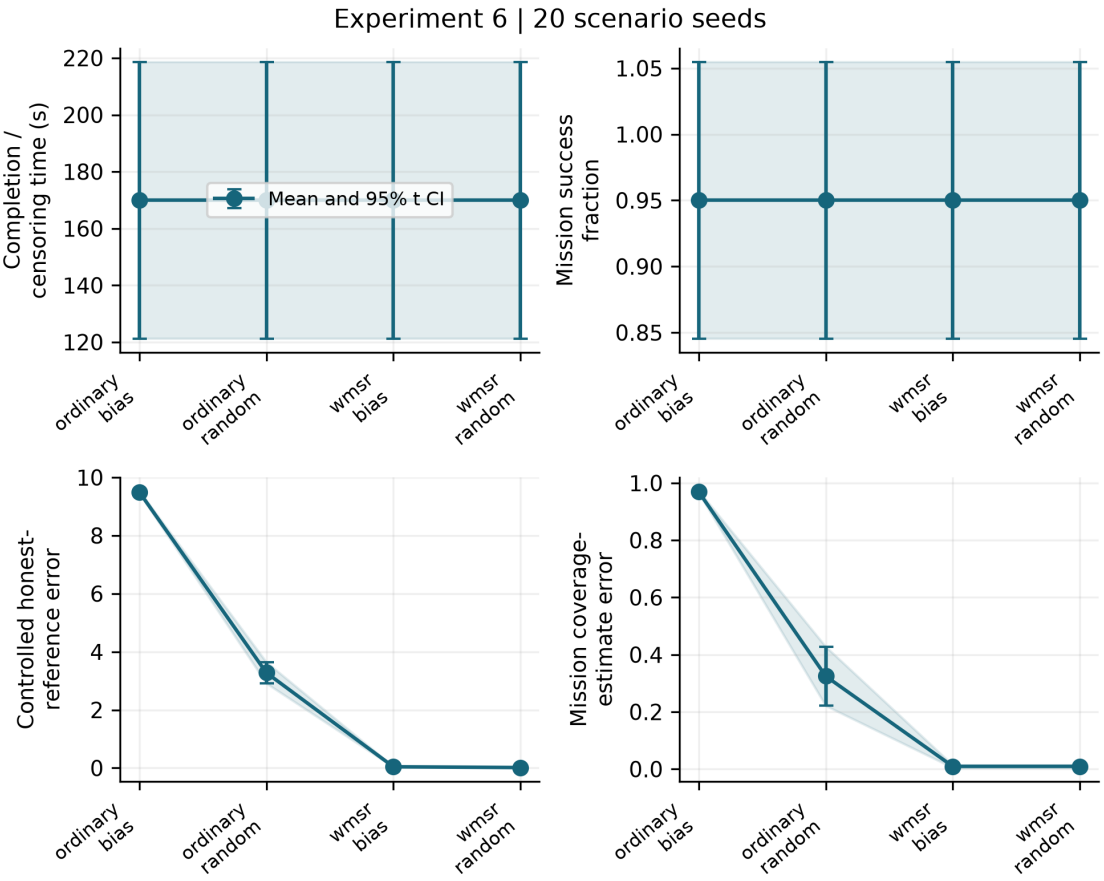



**Figure 6.6:** Faulty-report comparison. Controlled and mission estimate errors are separate measures. Source: Experiment 6 CSV.

## 6.8 Experiment 7. Ablations and sensitivity

The full configuration finished 18 of 20 missions. Disabling healing reduced that to six. Removing ADMM or resilient consensus left 18 successful missions in each condition. This suite did not apply the misinformation intervention of Experiment 6. The recorded timeout-3, timeout-10 and range-35 configurations each finished 19 missions. Both tested consensus relaxation variants finished 18. The small samples and configuration differences described in Chapter 5 limit causal conclusions for parameter variants. Figure 6.7 shows all nine conditions.

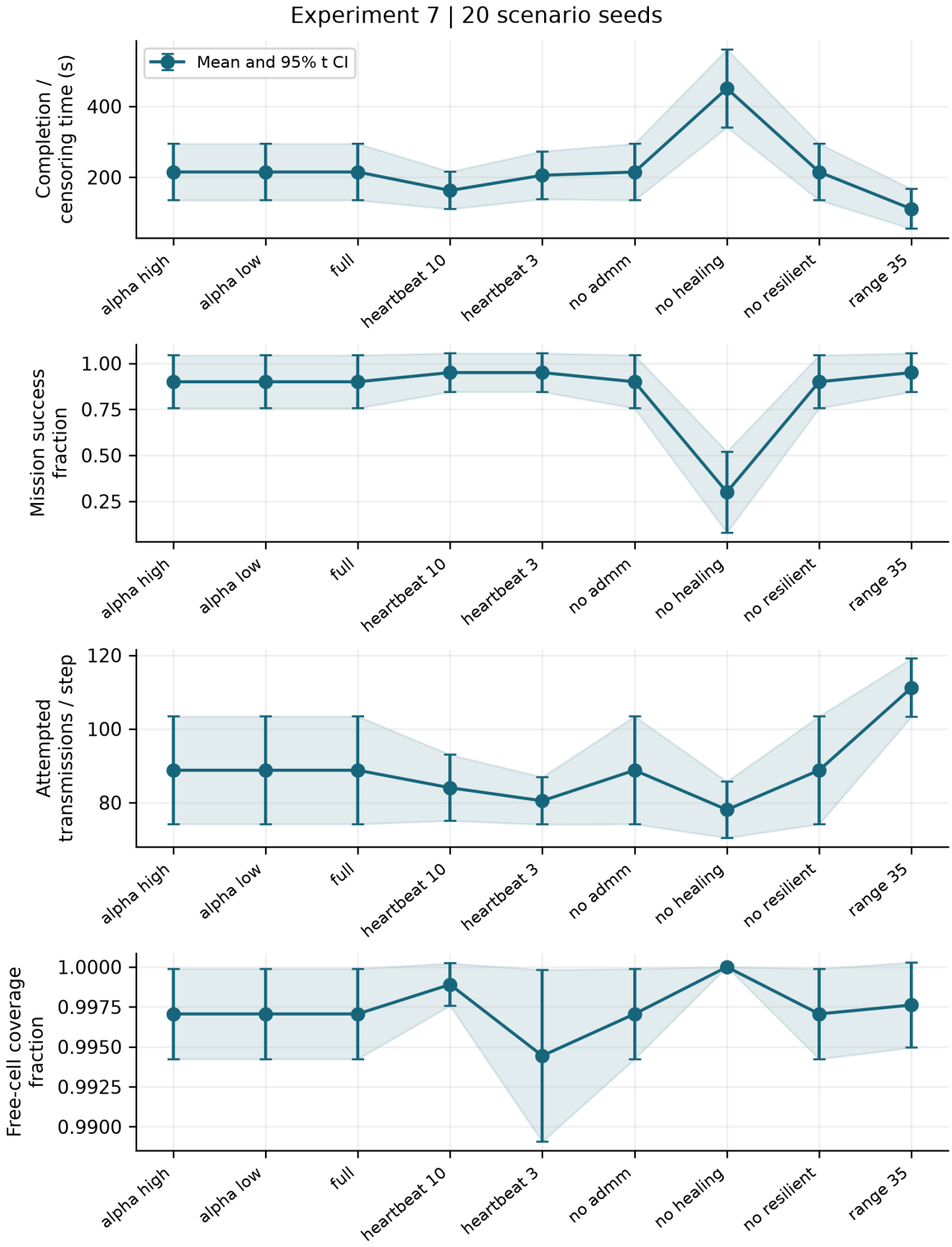



**Figure 6.7:** Ablation and sensitivity. Each point summarizes 20 runs. Source: Experiment 7 CSV.

## 6.9 Verification and an observed handoff

The full evaluation recorded zero pair-step separation violations and zero duplicate task execution steps. The smallest separation margin was about 0.050 units. Some responses still missed their deadlines. The saved verification record reports a passing completion gate with the strict recovery exception documented separately. Conditional separation depends on the accurate local proximity measurements and bounded speed specified in the model.

In the selected replay (Experiment 2, seed 3004), Drone 6 crashed at 21 seconds with survivor task S4 unfinished. A neighbor detected the failure at 24 seconds and a live drone resumed S4 at 40 seconds, a 19-second strict handoff. The replay records 20 losses of previously visible radio edges before the handoff while their endpoints remained in range. The second crash had no owned work. Figures A.6 and A.5 show this sequence. The original MP4 replay and animated GIF accompany the data. The seed illustrates one handoff.

## 6.10 Summary

Healing improved mission completion under injected crashes, while the strict reactivation check failed. The controlled tests show agreement on assignment cost and more reliable estimation under faulty reports. In complete missions, radio degradation, frequent ADMM fallback and incomplete rescues at larger populations remained material constraints.

Chapter 7

# Conclusion and Next Steps

## 7.1 Overview

We built and evaluated a swarm that searches, communicates and reallocates work when drones and links fail. The evidence supports continued operation after some failures and also identifies scenarios in which that coordination is insufficient.

## 7.2 What we learned

The clearest mission result was the improvement from 20% to 80% success with healing in the matched 20% crash comparison. Strict recovery instead measured a slight decline. The trace audit showed that a region can be searched by other drones without the failed drone's task being explicitly reactivated. Reporting both measures makes the limits of the claim clear.

Our distributed optimizer solved a controlled assignment instance to the tested accuracy, but changing membership and short mission rounds caused frequent auction fallback. W-MSR reduced estimate error under controlled scalar faults. The recorded motion and lease rules prevented separation and duplicate-execution violations in the evaluated cases. These checks are tied to the simulated observation, speed and voting assumptions.

## 7.3 Next steps

First, we would trace each unfinished rescue in the 100-drone cases through its lease, chosen motion and dwell history. That would distinguish lack of assignment from an assigned drone failing to arrive or remain in range. Second, we would measure whether longer or more stable optimization rounds reduce ADMM fallback enough to justify their communication and delay. Third, we would add sensing error and report bytes transmitted alongside packet attempts. A separate task selection rule could make priority influence which work begins first. A weight sweep could then assess the trade-offs in a new evaluation.

## 7.4 Summary

Local coordination can keep the simulated mission progressing after partial failure. Completing rescues under severe radio loss and at larger scale remains the main design challenge. The appendix gives the recorded diagnostics and the steps to reproduce the analysis.

Appendix A

# Reproduction and Supporting Figures

## A.1 Overview

This appendix locates the evidence behind the report and collects controlled diagnostics and selected replay views. All plots are original simulator outputs. The experiment summaries use 20 evaluation seeds per condition.

## A.2 Reproduction

The public repository contains the simulator and recorded runs. From its root, create an isolated Python environment and use the following commands.

```
python -m pip install -r requirements.txt
python -m pytest -q
python -m swarm.experiments.run_all --demo
python -m swarm.experiments.run_all --workers 4
python -m swarm.experiments.verify
```

The run directory contains one JSON per episode, including the configuration, history and source fingerprint. The full manifest records about 58.7 minutes for the 580 jobs on four workers, excluding final media rendering. The recorded cold demo took about 149 seconds including media. File paths in Table A.1 refer to the repository root.

**Table A.1:** Where to find the report evidence.

| Item | Location |
|---|---|
| Per-seed results and statistical summaries | `results/full/exp1.csv` through `results/full/exp7.csv`, and `results/full/summary.csv` |
| Full job histories and run timing | `results/full/jobs/`, `results/full/run_manifest.json` |
| Recovery investigation | `docs/INVESTIGATIONS.md`, `results/full/recovery_event_summary.csv` |
| Verification and assumptions | `results/sanity_report.md`, `docs/ASSUMPTIONS.md` |
| Replay and source figures | `results/animations/`, `results/figures/` |

## A.3 Controlled diagnostics

These figures test individual mechanisms. The ADMM objective gap concerns the fixed connected reference instance. The spectral plot compares the measured disagreement decay with

the fixed-graph prediction. During temporary disconnection the network cannot exchange information across components. The W-MSR curves use the controlled seven-node fault setting.

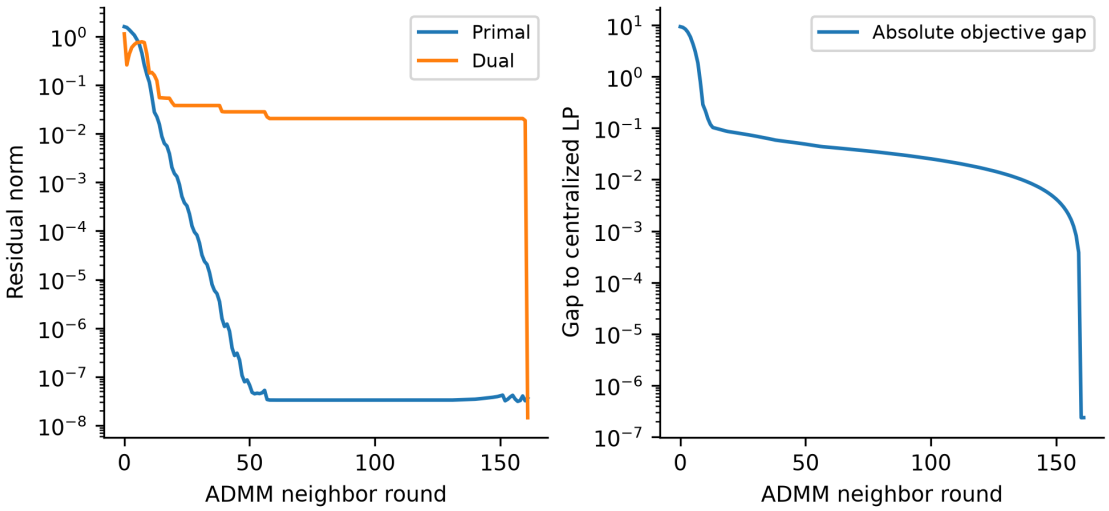



**Figure A.1:** Primal and dual residuals and objective gap against the centralized LP on a fixed connected instance.

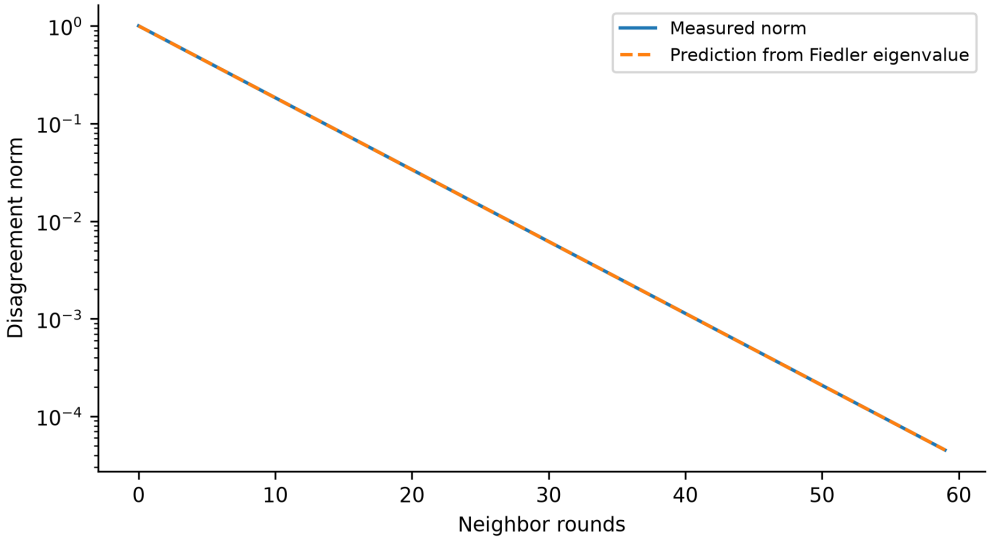



**Figure A.2:** Fixed-graph consensus disagreement and spectral prediction. The curves overlap at the plotted scale.

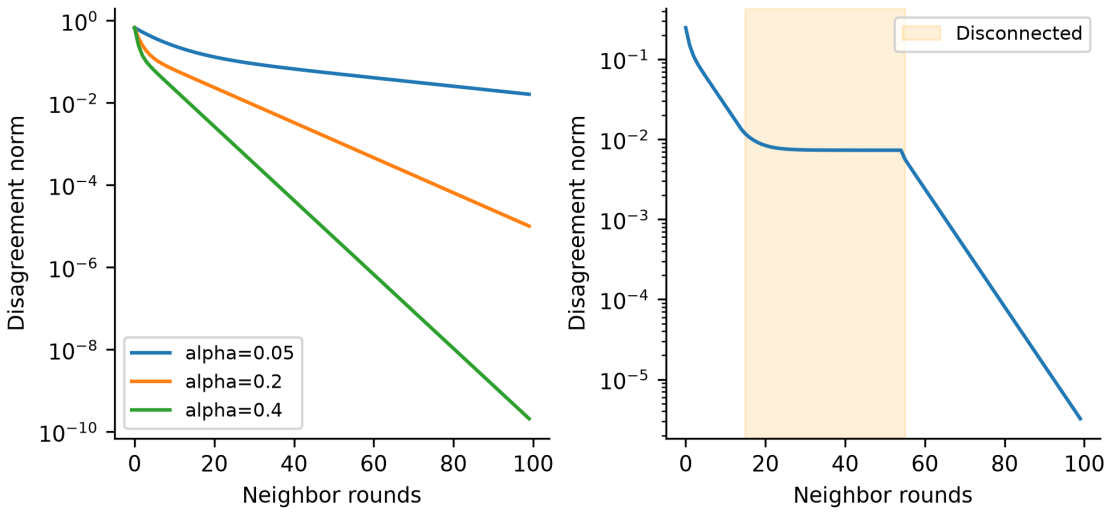



**Figure A.3:** Consensus step-size comparison and a controlled interval of temporary graph disconnection.

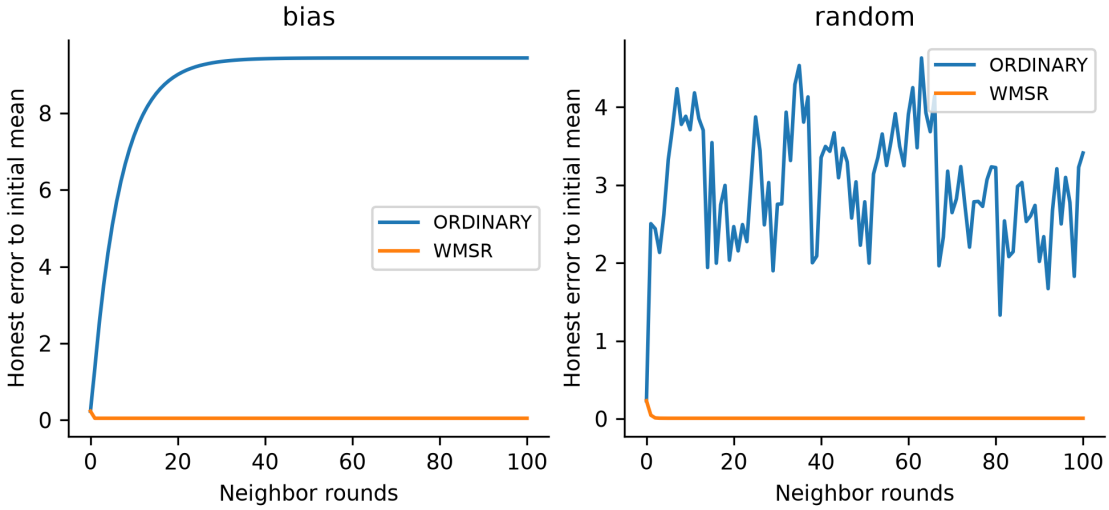



**Figure A.4:** Controlled error under biased and random reports. W-MSR keeps the honest estimates close in this tested setting.

## A.4 Recovery and safety views

The following snapshots and traces come from the selected mission. The heartbeat sweep illustrates false suspicions and missed failures in the replay. False positives count receiver suspicion events, while false negatives count physical crashes never detected. They have different denominators.

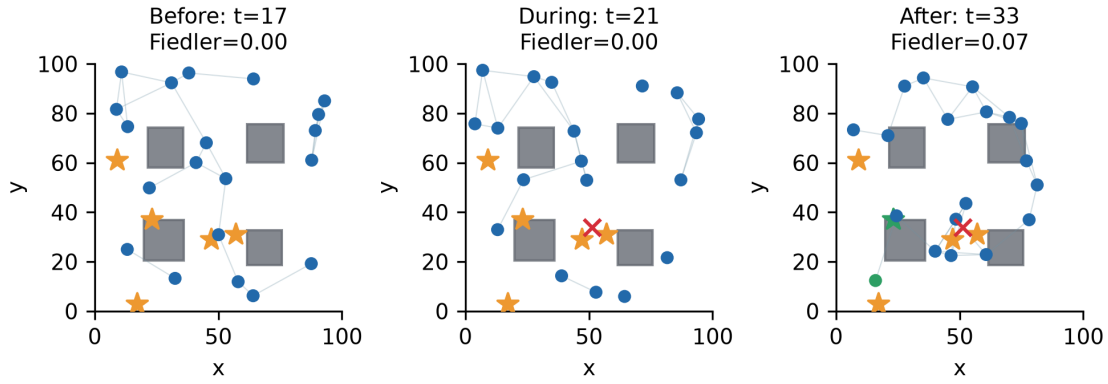



**Figure A.5:** Radio edges before, during and after the first failure. A red cross marks the crashed drone.

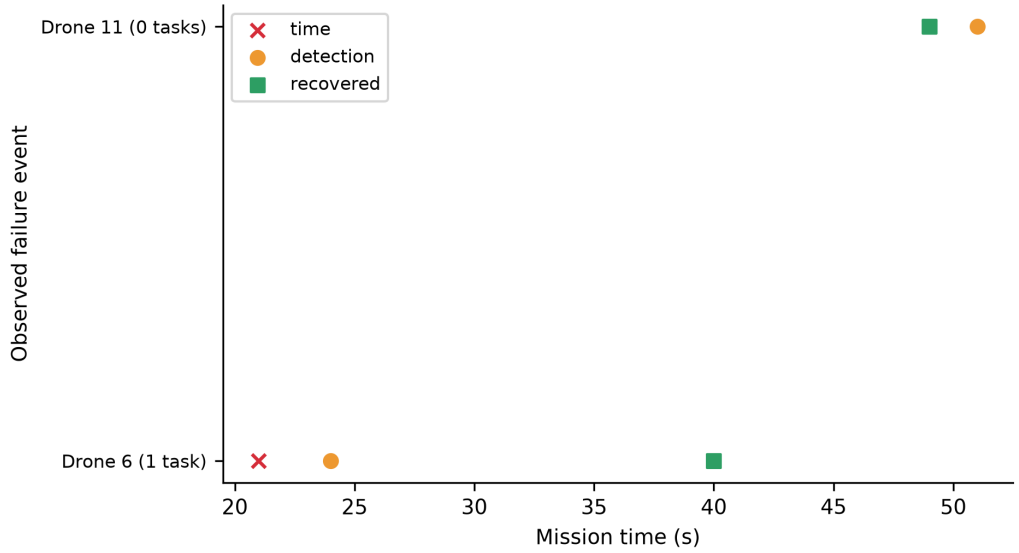



**Figure A.6:** Failure, detection and resumption on the replay timeline. The second failure had no unfinished task.

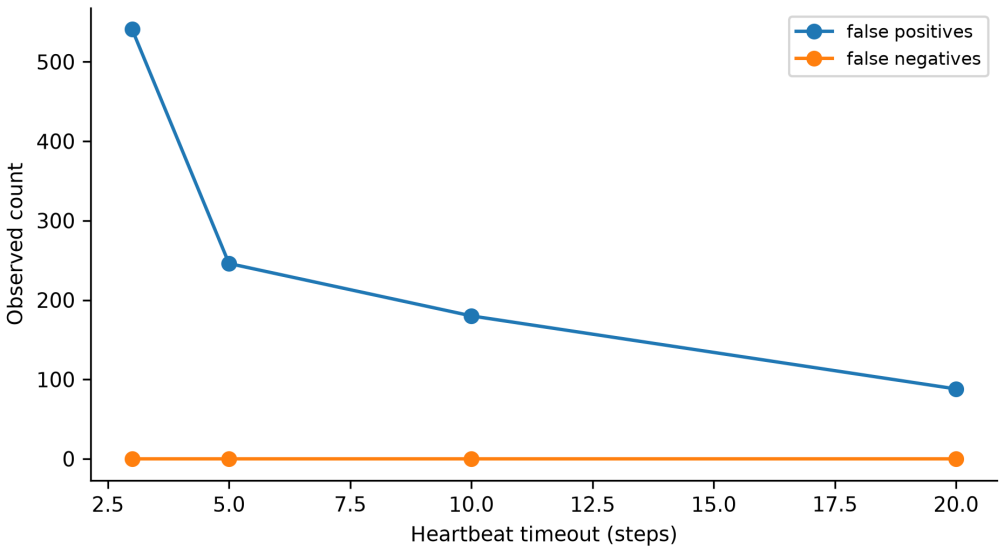



**Figure A.7:** Heartbeat false-positive and false-negative counts as the timeout changes on the recorded trace.

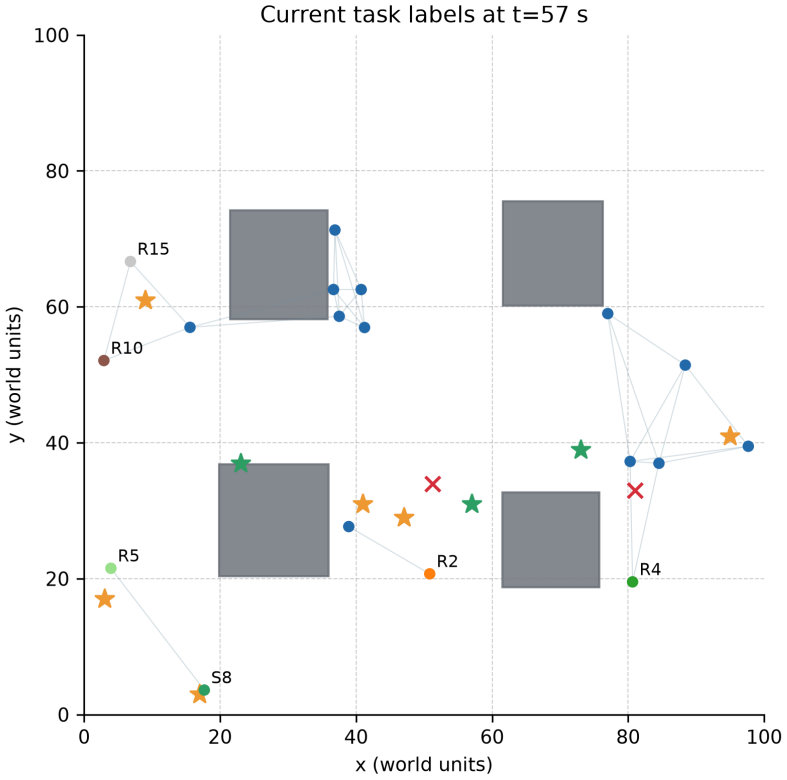



**Figure A.8:** Selected task labels on the recorded map at 57 seconds.

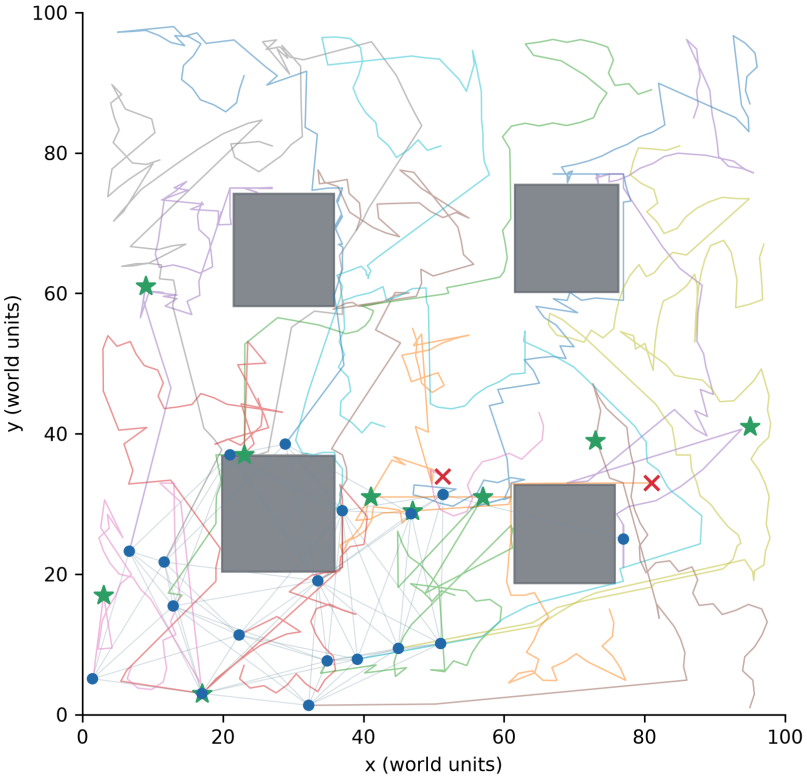



**Figure A.9:** Recorded flight trajectories around obstacles. Crosses show physical crashes.

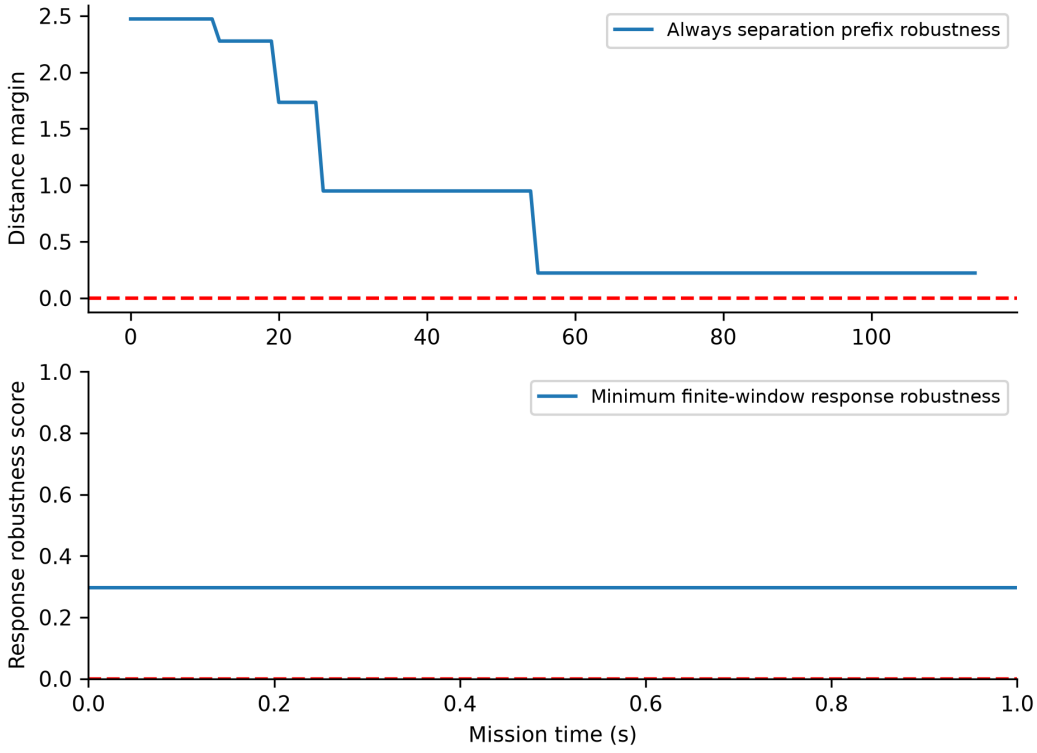



**Figure A.10:** Separation margin and the recorded response-window robustness for the selected mission. The dashed line marks zero.

## A.5 Summary across conditions

Figure A.11 summarizes mission success across the seven suites. Each condition has a different purpose. Use the within-suite comparisons in Chapter 6 when interpreting differences.

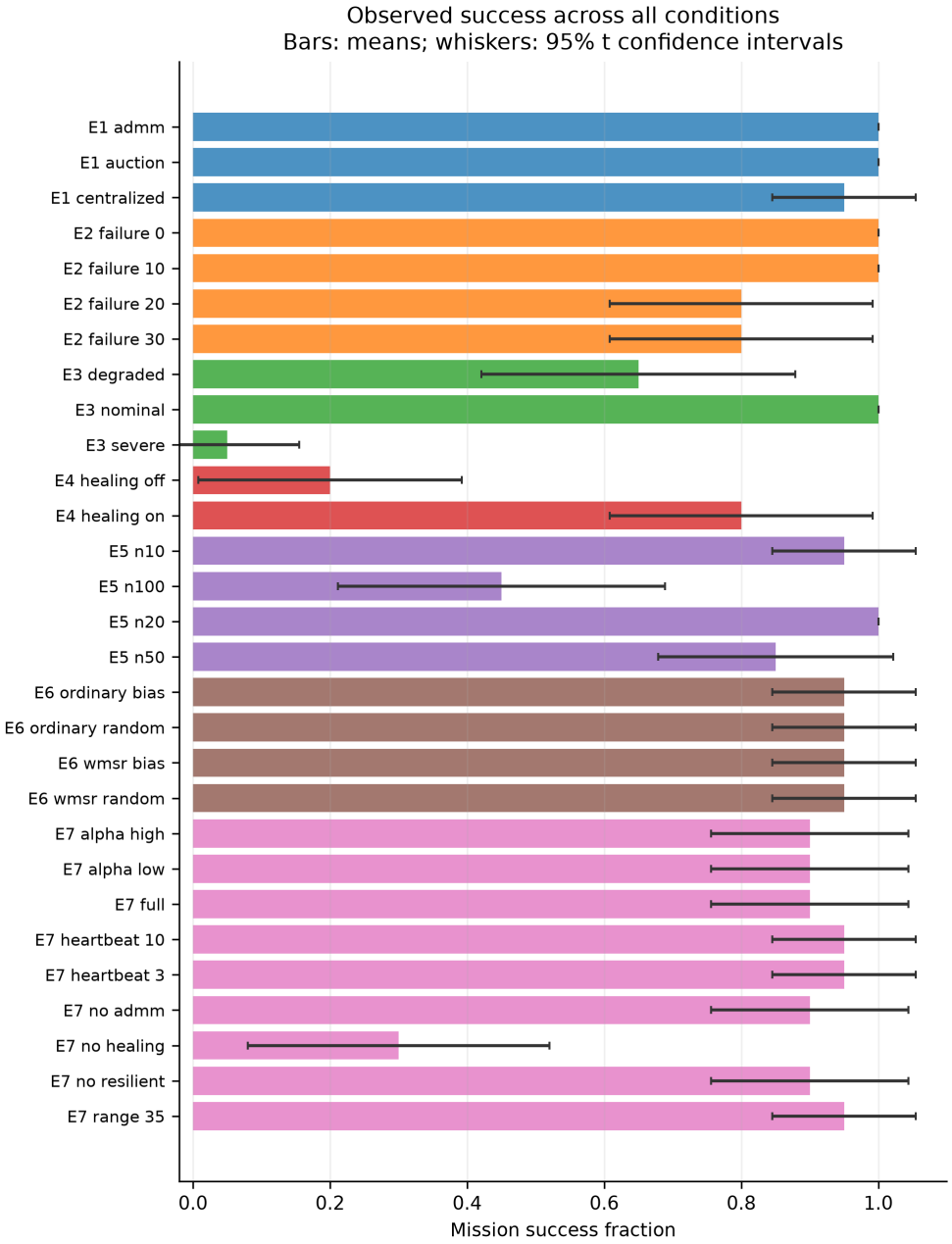



**Figure A.11:** Observed mission success across all 29 conditions, with approximate 95% Student $t$ intervals.

## A.6 Summary

The repository pairs source code with per-run evidence, aggregate tables and the original figures. The diagnostics and replay make specific claims inspectable alongside the full experiment results.

# References


[1] P. Dinesh, S. Dhanush, Ajay, and N. Kalyan. Self-healing distributed multi-agent drone swarm for disaster response. AI3403 group project, Indian Institute of Technology Hyderabad, 2026. Public repository, source, data and media.

[2] B. P. Gerkey and M. J. Matarić. A formal analysis and taxonomy of task allocation in multi-robot systems. *International Journal of Robotics Research*, 23(9), 939–954, 2004. doi:10.1177/0278364904045564.

[3] S. Boyd, N. Parikh, E. Chu, B. Peleato, and J. Eckstein. Distributed optimization and statistical learning via the alternating direction method of multipliers. *Foundations and Trends in Machine Learning*, 3(1), 1–122, 2011. doi:10.1561/2200000016.

[4] L. Xiao and S. Boyd. Fast linear iterations for distributed averaging. *Systems & Control Letters*, 53(1), 65–78, 2004. doi:10.1016/j.sysconle.2004.02.022.

[5] L. Xiao, S. Boyd, and S.-J. Kim. Distributed average consensus with time-varying Metropolis weights. Manuscript, June 2006. Stanford University project page.

[6] H. J. LeBlanc, H. Zhang, X. D. Koutsoukos, and S. Sundaram. Resilient asymptotic consensus in robust networks. *IEEE Journal on Selected Areas in Communications*, 31(4), 766–781, 2013. doi:10.1109/JSAC.2013.130413.

[7] G. E. Fainekos and G. J. Pappas. Robustness of temporal logic specifications for continuous-time signals. *Theoretical Computer Science*, 410(42), 4262–4291, 2009. doi:10.1016/j.tcs.2009.06.021.